SL22055 433.9 MHz Weather Sensor Protocol (Empirical Notes)

Reverse engineered from rtl_433 captures. Date: 2026-02-23.

Prepared for implementation (ESP + CC1101) and future reference.

1. Overview

This document describes an 8-byte (64-bit) OOK/PWM protocol observed from a 433.9 MHz weather sensor PCB marked SL22055. The goal is to reliably generate frames (ESP32 + CC1101) that a paired base station accepts, primarily for temperature (optionally humidity).

2. Capture setup and RF parameters

Captures were done using rtl_433 with a custom OOK PWM decoder.

Observed modulation/decoder parameters: OOK PWM, short ≈ 508 us, long ≈ 1008 us, frame gap ≈ 3012 us, reset ≈ 5024 us. Frequency used: 433.90 MHz.

```
rtl_433 -f 433.90M -s 2048000 -g 0 -R 0 \
  -X 'n=SWS001_like,m=OOK_PWM,s=508,l=1008,r=5024,g=0,t=0,y=3012' \
  -F json | jq -r 'select(.rows and .rows[0].len==64)'
```

3. Definitions (byte/bit order)

On-air bit order is effectively LSB-first within each byte. It is easiest to work in a “logical” byte view where each byte is bit-reversed.

```
Define bytes as B0..B7 (raw). Define logical bytes L[i] = bitrev8(B[i]).
bitrev8 reverses bit order within a byte:
  0babcdefgh -> 0bhgfedcba
```

4. Frame layout (64-bit / 8 bytes)

Each message is exactly 64 bits (8 bytes). Hex is shown MSB-first per byte (normal hex string).

Index	Byte	Meaning
0	B0	0xA5 (fixed header)
1	B1	0xD2 (fixed header)
2	B2	0xFD (fixed header)
3	B3	Temperature (bit-reversed)
4	B4	Device/flags (and repeat index in burst mode)
5	B5	Humidity (bit-reversed)
6	B6	Page/type (cycles)
7	B7	Check byte

5. Field decoding

Summary: the frame has a fixed 3-byte header, then payload bytes, then a check byte.

5.3 Page/type (B6)

Observed page/type byte values: 0x03, 0x83, 0x43, 0xC3, and a special 0xFD that appears in the cycle.

Logical mapping for the 4 normal pages: L6 = bitrev8(B6) = 0xC0 | page, where page is 0..3.

bitrev8(0xFD) = 0xBF (special / service subframe).

B6 (raw)	L6 = bitrev8(B6)	page
0x03	0xC0	0
0x83	0xC1	1
0x43	0xC2	2
0xC3	0xC3	3
0xFD	0xBF	(service)

5.4 B4 (device/flags and repeat index)

In periodic transmissions, B4 behaves like a sensor identifier / flags byte (keep it identical to the paired sensor).

In button-press mode the sensor sends a burst of 7 frames, and B4 encodes the repeat index rep=1..7.

```
Burst-mode repeat index (empirical):
  B4 = 0x90 | (bitrev3(rep) << 1)     where rep in [1..7]
After bit-reverse:
  L4 = bitrev8(B4) = (rep << 4) | 0x09
```

5.1 Temperature (B3)

Let L3 = bitrev8(B3). Temperature uses a linear mapping in a signed “raw” space with wrap modulo 256:

Raw value U (signed integer) is related to temperature by: U = round(482 - 23.5 * TempC). The transmitted byte stores only (U mod 256) as L3.

```
Decode (needs unwrap):
  L3 = bitrev8(B3)
  choose U = L3 + 256*k (k can be negative) that is closest to previous U (continuity)
  TempC = (482 - U) / 23.5

Encode:
  U = round(482 - 23.5 * TempC)
  L3 = U & 0xFF
  B3 = bitrev8(L3)
```

5.2 Humidity (B5)

Let L5 = bitrev8(B5). Humidity is encoded as: RH = Base(page) - L5.

Empirically, Base depends on page. From the provided logs: page 0/1 use Base=244, page 2 uses Base=242. Page 3 was observed close to 244, but should be re-validated if critical.

page	B6	Base(page)
0	0x03	244
1	0x83	244
2	0x43	242
3	0xC3	≈244 (needs confirm)

```
Decode:
  L5 = bitrev8(B5)
  RH = Base(page) - L5

Encode:
  L5 = Base(page) - RH
  B5 = bitrev8(L5)
```

6. Check byte (B7) - not CRC

Byte B7 is a simple additive check in the logical (bit-reversed) domain, then bit-reversed back. It is not a CRC.

```
Let L[i] = bitrev8(B[i]) for i=0..7.

Then the check satisfies:
  L7 = (L0 + L1 + L2 + L3 + L4 + L5 + L6 + 0x06) & 0xFF

Equivalently in raw bytes:
  B7 = bitrev8( (sum(bitrev8(B0..B6)) + 0x06) & 0xFF )
```

7. Transmitting spoofed frames

To spoof: take one known-good template frame captured from the paired sensor, update B3/B5, recompute B7, and transmit using the same OOK/PWM timings. If needed, repeat frames (burst) the same way the sensor does.

Recommended workflow: (1) capture a frame that the base station definitely decodes; (2) treat it as a template; (3) modify only B3 and (optionally) B5; (4) recompute B7; (5) transmit with the same OOK/PWM timing; (6) optionally send a burst of 7 repeats (button-press style) for reliability.

If the station stops accepting frames, the usual culprit is B4 (sensor identifier/flags). Keep B4 identical to the paired sensor's accepted frames.

8. Worked examples

Examples below show decoding and encoding using frames from the provided logs.

```
Example A (cooling log):
  Frame: a5 d2 fd 30 5f 0b 83 32
  B3=0x30 -> L3=bitrev8(0x30)=0x0C=12
  TempC=(482-12)/23.5 = 20.0
  B5=0x0B -> L5=bitrev8(0x0B)=0xD0=208
  page=1 (B6=0x83) -> Base=244 -> RH=244-208=36
  Check byte verified by the additive rule.

Example B (heating log, wrap):
  Frame: ... B3=0xC0 (seen at ~31.3C)
  L3=bitrev8(0xC0)=0x03=3
  Candidates U = 3 + 256*k. With continuity from previous values, choose U=-253 (k=-1):
  TempC=(482-(-253))/23.5 = 31.28C (approx).
```

9. Reference code

Reference implementations are included (C/C++ style and Python).

```
// bit reverse within a byte
uint8_t bitrev8(uint8_t x){
  x = (x >> 4) | (x << 4);
  x = ((x & 0xCC) >> 2) | ((x & 0x33) << 2);
  x = ((x & 0xAA) >> 1) | ((x & 0x55) << 1);
  return x;
}

uint8_t checksum_B7(const uint8_t b[7]){
  uint16_t s = 0;
  for(int i=0;i<7;i++) s = (s + bitrev8(b[i])) & 0xFF;
  s = (s + 0x06) & 0xFF;
  return bitrev8((uint8_t)s);
}
```

```
def bitrev8(x: int) -> int:
    x = ((x & 0xF0) >> 4) | ((x & 0x0F) << 4)
    x = ((x & 0xCC) >> 2) | ((x & 0x33) << 2)
    x = ((x & 0xAA) >> 1) | ((x & 0x55) << 1)
    return x

def checksum_b7(b7bytes7: bytes) -> int:
    s = sum(bitrev8(b) for b in b7bytes7) & 0xFF
    s = (s + 0x06) & 0xFF
    return bitrev8(s)
```

10. Known unknowns / limitations

B4 meaning outside the observed repeat-index burst is not fully decoded. For best compatibility, keep B4 exactly as the paired sensor uses in accepted frames, except for the documented burst variant.